

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:

«Информационная безопасность баз данных»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

«Реализация сервиса для взаимодействия с разработанной базой данных»

Выполнили:

Шибает Иван Дмитриевич, студент группы N3352

(подпись)

Малышев Григорий Игоревич, студент группы
N3352

(подпись)

Проверил:

Ярцева Наталия Андреевна

(подпись)

СОДЕРЖАНИЕ

Введение	3
1 Выбор среды, языка и фреймворка	5
2 Структура и модули сервиса	6
2.1 Состав	6
2.2 Дополнения базы данных	6
2.3 Модуль аутентификации «auth»	7
2.4 Модуль доступа к данным «db»	8
2.5 Модуль отчётов «reports»	9
2.6 Приложение Flask	9
3 Взаимодействие с базой данных	11
3.1 Список таблиц по роли	11
3.2 Выборка, вставка, изменение и удаление	14
3.3 Сложные запросы по ролям	16
3.4 Регистрация пользователей	21
3.5 Контроль журналирования и разграничения доступа	22
4 Защита данных в сервисе	25
4.1 Реализованные меры	25
4.2 Аналоги, достоинства и недостатки	26
5 Инструкция по запуску	27
Заключение	28
Приложение А Скрипт базы данных и модули Python	29
Приложение Б Приложение Flask	42
Приложение В Шаблоны и стили	50
Приложение Г Сценарии проверки	57

ВВЕДЕНИЕ

Цель работы — получение навыков использования серверных языков программирования, фреймворков и средств доступа к базе данных.

Задание требует:

- создать сервис, взаимодействующий с базой данных из предыдущих работ, и кратко обосновать выбор среды разработки, языка и фреймворка;
- описать способ взаимодействия с базой данных, продемонстрировать выборку, вставку и удаление данных средствами выбранного фреймворка и проконтролировать работу журналирования и разграничения доступа, реализованных в работах № 1–3;
- указать основные функции, назначение и структуру сервиса, описать и оценить методы защиты данных, выделить аналоги, достоинства и недостатки решения.

Примечания к заданию требуют подбирать для сложных запросов отношения, подходящие роли пользователя, привести полный листинг кода интерфейса, достаточный для запуска, описать необходимую среду, а также назначение и алгоритм всех модулей.

В качестве сервиса выбран многопользовательский веб-интерфейс, как в примере из методических указаний. Требования к нему:

- вход под учётной записью хотя бы одного пользователя каждой роли из работ № 1–3 и под учётной записью администратора, которому доступны все таблицы;
- список таблиц формируется по роли учётной записи; данные можно просматривать, добавлять и удалять;
- для каждой группы пользователей выводится результат сложного запроса с группировкой («GROUP BY») по данным более чем двух сущностей с сортировкой и фильтрацией в интерфейсе;
- дополнительно — регистрация новых пользователей существующих ролей.

Работа выполнена на базе данных «workshop» мастерской реставрации музыкальных инструментов (PostgreSQL 14.21), созданной в работе №

2 и защищённой в работе № 3: журнал изменений «change_log», зашифрованная таблица «access_secret», групповые роли «reception_role» (приёмщик) и «restorer_role» (реставратор) со столбцовыми привилегиями и политиками строк.

1 Выбор среды, языка и фреймворка

Сервис написан на Python 3.13. Язык выбран потому, что для него есть зрелые драйвер PostgreSQL и лёгкие веб-фреймворки, а короткий код удобно приводить в отчёте полностью. Среда разработки не влияет на результат: использовались текстовый редактор и терминал macOS; для запуска достаточно интерпретатора Python и установленных по файлу «requirements.txt» пакетов в локальном окружении «.venv».

Использованы следующие библиотеки:

- Flask 3.1.3 — минимальный веб-фреймворк: маршруты, шаблоны Jinja2 с автоматическим экранированием, подписанная cookie сеанса. Возможностей хватает для серверного рендеринга страниц без JavaScript;
- psycopg 3.3.6 — драйвер PostgreSQL. Значения передаются серверу отдельно от текста запроса, а модуль «psycopg.sql» безопасно подставляет имена таблиц и столбцов;
- Playwright 1.63.0 — только для получения снимков экрана: управляет установленным Google Chrome 153 и в работе приложения не участвует.

ORM (например, SQLAlchemy из примера в методических указаниях) сознательно не используется. Набор доступных отношений и даже столбцов здесь не фиксирован, а определяется привилегиями вошедшей роли: реставратор видит только часть столбцов таблиц «instrument», «service», «employee», «restoration_order» и «order_work». Классы ORM описывают все столбцы и при чтении перечисляют их, поэтому такой запрос реставратора был бы отклонён СУБД. Прямая работа с драйвером и каталогом PostgreSQL позволяет строить запросы по фактическим правам роли и не дублировать схему в коде.

2 Структура и модули сервиса

2.1 Состав

Файлы работы и их назначение перечислены в таблице 1. Полные листинги приведены в приложениях А–Г.

Таблица 1 — Файлы сервиса

Файл	Назначение
sql/01_admin.sql	роль администратора, политики строк для неё, функция регистрации пользователей (листинг А.1)
webapp/auth.py	вход через СУБД, серверные сеансы, защита от подбора, CSRF-токен, SCRAM-верификатор (листинг А.2)
webapp/db.py	каталог доступных отношений и столбцов, выборка с фильтром и сортировкой, вставка, изменение, удаление (листинг А.3)
webapp/reports.py, webapp/sql/*.sql	сводные запросы с группировкой для каждой группы пользователей (листинги А.4, 1–4)
webapp/__init__.py	приложение Flask: маршруты, проверки сеанса и CSRF, заголовки безопасности (листинг Б.1)
webapp/__main__.py	запуск сервера на 127.0.0.1 (листинг Б.2)
webapp/templates/*.html, webapp/static/style.css	шаблоны страниц и стили (приложение В)
check.sh, check.py	пересборка БД и автоматическая проверка (приложение Г)
shots.py	снимки экрана работающего приложения (листинг Г.3)
requirements.txt, Makefile	зависимости и команды (листинги Б.3, Б.4)

Сервис устроен в три слоя. Нижний слой — PostgreSQL с ролями, привилегиями, политиками строк и журналом из работы № 3. Средний слой — модули «auth», «db» и «reports», которые открывают соединение от имени пользователя и строят запросы. Верхний слой — маршруты Flask и шаблоны страниц. Приложение не хранит собственных пользователей и ролей: каждый пользователь работает с СУБД через собственное соединение под своей учётной записью.

2.2 Дополнения базы данных

Скрипт «sql/01_admin.sql» (листинг А.1) выполняется суперпользователем после скриптов работы № 3 и создаёт:

1. групповую роль «admin_role» и учётную запись «workshop_admin». Роль не является суперпользователем: ей выданы все операции над

девятью основными таблицами, чтение четырёх представлений, а также только чтение журнала «change_log» и таблицы «access_secret». Подделать журнал или заменить шифртекст администратор не может, а секреты без индивидуального пароля остаются нечитаемыми. Этим расширяется правило работы № 3, по которому журнал читал только суперпользователь: теперь его может читать и администратор приложения;

2. политики «admin_all» на таблицах «order_work» и «inspection». На них включена защита строк, и без собственной политики администратор не увидел бы ни одной строки;
3. функцию «app_register_user» для регистрации пользователей. Алгоритм: проверить имя по шаблону (строчные латинские буквы, цифры, подчёркивание; не «pg_»), проверить, что группа — «reception_role» или «restorer_role», что передан SCRAM-верификатор и что роли с таким именем нет; выполнить «CREATE ROLE ... LOGIN ... IN ROLE ...» с подстановкой имён через «format(„%I“)»; пометить роль комментарием; для реставратора связать учётную запись с сотрудником через «employee.db_login». Функция объявлена «SECURITY DEFINER», право её выполнения есть только у «admin_role». Создать роль в группе администраторов или суперпользователя через неё нельзя.

2.3 Модуль аутентификации «auth»

Модуль (листинг A.2) реализует вход и сеансы. Алгоритм входа (функция «login»):

1. проверить, не заблокирована ли пара «имя, адрес клиента»: после пяти неудач за пять минут вход временно отклоняется;
2. проверить имя по шаблону и открыть соединение «psycopg.connect» с именем и паролем из формы. Пароль проверяет PostgreSQL по протоколу SCRAM-SHA-256; при ошибке выдаётся одинаковое сообщение «Неверное имя пользователя или пароль», чтобы не раскрывать, существует ли роль;
3. функцией «check_account» убедиться, что при подключении действительно использовался пароль и что роль не является суперпользователем. На учебном кластере «postgres» входит с

127.0.0.1 без пароля (метод trust), и без этой проверки любой пароль открыл бы через веб-интерфейс сеанс суперпользователя;

4. определить вид интерфейса по членству в групповой роли («pg_has_role»): администратор, приёмщик или реставратор. Роль без такой группы не допускается;
5. установить «statement_timeout» и сохранить сеанс в памяти процесса под случайным идентификатором из 32 байт; в cookie попадает только этот идентификатор.

Класс «SessionStore» хранит сеансы и неудачные попытки. Сеанс истекает через 15 минут бездействия или 8 часов после входа; при истечении соединение с СУБД закрывается. Функции «new_csrf» и «csrf_ok» создают и сравнивают CSRF-токен за постоянное время. Функция «scram_verifier» по RFC 5802 и RFC 7677 вычисляет из пароля строку «SCRAM-SHA-256\$4096:соль\$StoredKey:ServerKey»: 4096 итераций PBKDF2-HMAC-SHA-256 со случайной солью, затем HMAC-ключи клиента и сервера. PostgreSQL принимает такую строку в «CREATE ROLE ... PASSWORD» и сохраняет как есть, поэтому при регистрации пароль не передаётся серверу в открытом виде.

2.4 Модуль доступа к данным «db»

Модуль (листинг A.3) выполняет все обращения к таблицам:

- «relations» — запрос к каталогу «pg_class»: отношения схемы «public», на которые у текущей роли есть право чтения хотя бы одного столбца, и признаки прав на вставку, изменение и удаление. Используются функции «has_any_column_privilege» и «has_table_privilege»;
- «columns» — столбцы отношения из «pg_attribute» с признаками прав на чтение, вставку и изменение каждого столбца («has_column_privilege»), наличием значения по умолчанию и принадлежностью к первичному ключу («pg_index»);
- «select_rows» — выборка только читаемых столбцов с необязательным фильтром («содержит» через «LIKE» с экранированием «%» и «_» или «равно») и сортировкой, не более 200 строк;

- «insert_row», «delete_row», «update_value» — вставка заполненных полей, удаление и изменение одного значения по первичному ключу.

Имена отношений и столбцов попадают в текст запроса только после сверки со списками из каталога и только через «sql.Identifier», а значения — только параметрами. Права на операции модуль не проверяет: решение принимает СУБД, а её ошибки показываются пользователю.

2.5 Модуль отчётов «reports»

Модуль (листинг А.4) описывает четыре сводных запроса: по одному для приёмщика и реставратора и два для администратора. Тексты запросов хранятся в отдельных файлах «webapp/sql/*.sql». Для каждого отчёта заданы параметры с конечным списком значений; значение вне списка заменяется пустой строкой («все»). Функция «columns» выполняет запрос с «LIMIT 0» и получает имена столбцов результата — они становятся белым списком для сортировки и фильтра. Функция «run» оборачивает запрос: «SELECT * FROM (...) r WHERE ... ORDER BY ...» и добавляет фильтр и сортировку так же, как для таблиц.

2.6 Приложение Flask

Функция «create_app» (листинг Б.1 в приложении Б) настраивает приложение и регистрирует маршруты из таблицы 2. Перед каждым запросом обработчик «load_user» находит сеанс по идентификатору из cookie и для любого POST-запроса сверяет CSRF-токен формы с токеном сеанса; при несовпадении запрос отклоняется с кодом 400. После каждого запроса «headers» добавляет заголовки безопасности. Декоратор «login_required» перенаправляет неавторизованного пользователя на страницу входа и сериализует обращения одного сеанса к его соединению. Ошибки СУБД перехватываются и показываются сообщением с кодом SQLSTATE.

Таблица 2 — Маршруты приложения

Маршрут	Метод	Действие
/login	GET, POST	форма входа; вход через СУБД
/logout	POST	завершение сеанса, закрытие соединения

Маршрут	Метод	Действие
/tables	GET	список доступных отношений и отчётов
/t/⟨имя⟩	GET	просмотр с фильтром и сортировкой, формы вставки, изменения и удаления
/t/⟨имя⟩/insert	POST	вставка строки
/t/⟨имя⟩/update	POST	изменение значения по ключу
/t/⟨имя⟩/delete	POST	удаление строки по ключу
/r/⟨отчёт⟩	GET	сводный отчёт группы пользователя
/register	GET, POST	список пользователей и регистрация (администратор)

Шаблоны (приложение В) наследуют общий «base.html» с меню и кнопкой выхода. Шаблон «_filter.html» — общая форма фильтра и сортировки, «table.html» выводит таблицу и формы изменения данных только при наличии соответствующей привилегии, «report.html» — отчёт с его параметрами, «register.html» — регистрацию. JavaScript не используется.

3 Взаимодействие с базой данных

3.1 Список таблиц по роли

Список таблиц не задан в коде: страница «/tables» выполняет запрос из функции «relations» от имени вошедшего пользователя. Результаты для трёх ролей, сохранённые проверкой, приведены в таблицах 3, 4 и 5 (S — чтение всех столбцов, S* — части столбцов, I — вставка, U — изменение, D — удаление). Они совпадают с матрицей привилегий работы № 3: приёмщику доступны 8 отношений, реставратору 9, администратору 15, включая журнал и таблицу секретов. Страницы приёмщика и реставратора показаны на рисунках 1 и 2.

Таблица 3 — Отношения приёмщика «reception_orlova»

Отношение	Вид	Чтение	Вставка	Изменение	Удаление
customer	TABLE	S	I	U	
instrument	TABLE	S	I	U	
instrument_type	TABLE	S			
payment	TABLE	S	I	U	
restoration_order	TABLE	S	I	U	
service	TABLE	S			
v_reception_balances	VIEW	S			
v_reception_orders	VIEW	S			

Таблица 4 — Отношения реставратора «restorer_smirnov»

Отношение	Вид	Чтение	Вставка	Изменение	Удаление
employee	TABLE	S*			
inspection	TABLE	S	I		
instrument	TABLE	S*			
instrument_type	TABLE	S			
order_work	TABLE	S*		U	
restoration_order	TABLE	S*			
service	TABLE	S*			
v_restorer_inspections	VIEW	S			
v_restorer_tasks	VIEW	S			

Таблица 5 — Отношения администратора «workshop_admin»

Отношение	Вид	Чтение	Вставка	Изменение	Удаление
access_secret	TABLE	S			
change_log	TABLE	S			

Отношение	Вид	Чтение	Вставка	Изменение	Удаление
customer	TABLE	S	I	U	D
employee	TABLE	S	I	U	D
inspection	TABLE	S	I	U	D
instrument	TABLE	S	I	U	D
instrument_type	TABLE	S	I	U	D
order_work	TABLE	S	I	U	D
payment	TABLE	S	I	U	D
restoration_order	TABLE	S	I	U	D
service	TABLE	S	I	U	D
v_reception_balances	VIEW	S			
v_reception_orders	VIEW	S			
v_restorer inspections	VIEW	S			
v_restorer_tasks	VIEW	S			

Доступные отношения

Список построен по привилегиям роли reception_orlova в СУБД. S* — чтение части столбцов.

Отношение	Вид	Чтение	Вставка	Изменение	Удаление
customer	TABLE	S	I	U	
instrument	TABLE	S	I	U	
instrument_type	TABLE	S			
payment	TABLE	S	I	U	
restoration_order	TABLE	S	I	U	
service	TABLE	S			
v_reception_balances	VIEW	S			
v_reception_orders	VIEW	S			

Отчёты

- [Оплаты по типам инструментов](#) — Заказчики, заказы и поступившие оплаты по типам инструментов (instrument_type, instrument, restoration_order, payment).

Рисунок 1 — Главная страница приёмщика

Доступные отношения

Список построен по привилегиям роли restorer_smirnov в СУБД. S* — чтение части столбцов.

Отношение	Вид	Чтение	Вставка	Изменение	Удаление
employee	TABLE	S*			
inspection	TABLE	S	I		
instrument	TABLE	S*			
instrument_type	TABLE	S			
order_work	TABLE	S*		U	
restoration_order	TABLE	S*			
service	TABLE	S*			
v_restorer_inspections	VIEW	S			
v_restorer_tasks	VIEW	S			

Отчёты

- [Работы по исполнителям и услугам](#) — Число работ по статусам и ближайший срок (order_work, employee, service, restoration_order).

Рисунок 2 — Главная страница реставратора

Главная страница администратора со всеми отношениями базы данных показана на рисунке 3.

Доступные отношения

Список построен по привилегиям роли workshop_admin в СУБД. S* — чтение части столбцов.

Отношение	Вид	Чтение	Вставка	Изменение	Удаление
access_secret	TABLE	S			
change_log	TABLE	S			
customer	TABLE	S	I	U	D
employee	TABLE	S	I	U	D
inspection	TABLE	S	I	U	D
instrument	TABLE	S	I	U	D
instrument_type	TABLE	S	I	U	D
order_work	TABLE	S	I	U	D
payment	TABLE	S	I	U	D
restoration_order	TABLE	S	I	U	D
service	TABLE	S	I	U	D
v_reception_balances	VIEW	S			
v_reception_orders	VIEW	S			
v_restorer_inspections	VIEW	S			
v_restorer_tasks	VIEW	S			

Отчёты

- [Выработка сотрудников](#) — Заказы, заказчики, работы и согласованная стоимость по сотрудникам (employee, order_work, restoration_order, instrument).
- [Журнал изменений по ролям](#) — Операции из change_log по группам, ролям входа и таблицам (change_log, pg_roles, pg_auth_members).

Рисунок 3 — Главная страница администратора

3.2 Выборка, вставка, изменение и удаление

Выборка выполняется функцией «select_rows». На рисунке 4 приёмщик отфильтровал заказчиков по вхождению «ова» в «full_name» и отсортировал их по убыванию; ниже расположены формы вставки и изменения, а кнопок удаления нет — у роли нет права «DELETE». При чтении таблицы, где роли доступна только часть столбцов, выводятся только они, а скрытые перечисляются (рисунок 5: реставратору не показывается связь инструмента с заказчиком «customer_id»).

customer

Фильтр

full_name

▼

содержит

ова

▼

Сортировка

full_name

▼

по убыванию

▼

Показать

Строк: 4 (не более 200).

customer_id	full_name	phone	email
7	Фёдорова Мария Константиновна	+7 900 000-00-07	fedorova@example.test
5	Морозова Ирина Алексеевна	+7 900 000-00-05	irina@example.test
1	Кузнецова Анна Сергеевна	+7 900 000-00-01	anna@example.test
9	Белова Ксения Олеговна	+7 900 000-00-09	belova@example.test

Добавить строку

customer_id

integer, по умолчанию

full_name

text

phone

text

email

text

Добавить

Изменить значение

customer_id (ключ)

customer_id

▼

Новое значение

Изменить

Рисунок 4 — Таблица «customer» у приёмщика: фильтр и сортировка

instrument

Фильтр

—

▼

содержит

▼

Сортировка

—

▼

по возрастанию

▼

Показать

Строк: 9 (не более 200). Скрыты столбцы без права чтения: customer_id.

instrument_id	type_id	inventory_no	name	maker	serial_no
1	1	INS-001	Скрипка ученическая 4/4	Cremona	
2	2	INS-002	Гитара классическая	Yamaha	DEMO-C40
3	3	INS-003	Альт концертный 16"	Hofner	DEMO-H16
4	4	INS-004	Виолончель 4/4	Strunal	
5	5	INS-005	Контрабас оркестровый 3/4	Cremona	DEMO-SB3
6	6	INS-006	Мандолина неаполитанская		
7	7	INS-007	Балалайка прима	Мастер Е. Зув	
8	8	INS-008	Домра малая		DEMO-D01
10	2	INS-009	Гитара концертная		

Рисунок 5 — Таблица «instrument» у реставратора: скрытый столбец

Операции изменения данных проверены сценарием «check.py» (листинг Г.2): приёмщик добавил заказчика, реставратор изменил

статус своей работы и зарегистрировал осмотр, администратор удалил добавленного заказчика. Запрещённые операции отклонила СУБД: удаление заказчика приёмщиком — ошибкой «permission denied» (SQLSTATE 42501), изменение чужой работы не затронуло ни одной строки, а осмотр от имени другого сотрудника отклонён политикой строк. На рисунке 6 показано последнее сообщение в интерфейсе.

Мастерская · БД workshop
Таблицы
Работы по исполнителям и услугам
restorer_smirnov (реставратор)
Выйти

Ошибка СУБД (42501): new row violates row-level security policy for table "inspection"

inspection

Фильтр

—

содержит

Сортировка

—

по возрастанию

Показать

Строк: 10 (не более 200).

inspection_id	order_id	employee_id	inspected_on	conclusion
1	1	1	2026-09-01	Продолжая тренинж, руки длиной 25 см

Рисунок 6 — Осмотр от чужого имени отклонён политикой строк

3.3 Сложные запросы по ролям

Для каждой группы пользователей составлен запрос с группировкой по данным не менее чем трёх сущностей; использованы только отношения и столбцы, доступные этой роли. Отчёт открывается из меню, параметры отчёта задаются списками, а по любому столбцу результата можно отфильтровать строки и отсортировать их.

Приёмщик. Отчёт «Оплаты по типам инструментов» (листинг 1) соединяет «instrument_type», «instrument», «restoration_order» и «payment» и для каждого типа инструмента считает заказчиков, заказы, число и сумму оплат и дату последней оплаты. Параметры — статус заказа и способ оплаты. Таблица «order_work» с ценами работ приёмщику недоступна и в запросе не используется. Результат с сортировкой по сумме показан на рисунке 7, результат с фильтром «способ оплаты card», условием «instrument_type содержит «к»» и сортировкой по типу — в таблице 6.

```
-- Приёмщик: оплаты по типам инструментов.
SELECT t.name AS instrument_type,
       count(DISTINCT i.customer_id) AS customers,
       count(DISTINCT o.order_id) AS orders,
       count(p.payment_id) AS payments,
```

16

```

        round(coalesce(sum(p.amount_kopecks), 0) / 100.0, 2)
        AS paid_rub,
        max(p.paid_on) AS last_paid_on
FROM instrument_type t
JOIN instrument i ON i.type_id = t.type_id
JOIN restoration_order o
    ON o.instrument_id = i.instrument_id
LEFT JOIN payment p ON p.order_id = o.order_id
    AND (method)s = '' OR p.method = (method)s
WHERE (ostatus)s = '' OR o.status = (ostatus)s
GROUP BY t.name

```

Листинг 1 — Отчёт приёмщика («webapp/sql/payments.sql»)

Мастерская · БД workshop

[Таблицы](#)
[Оплаты по типам инструментов](#)

reception_orlova (приёмщик) [Выйти](#)

Оплаты по типам инструментов

Заказчики, заказы и поступившие оплаты по типам инструментов (instrument_type, instrument, restoration_order, payment).

Статус заказа

Способ оплаты

Фильтр

Сортировка

по убыванию

Показать

instrument_type	customers	orders	payments	paid_rub	last_paid_on
Альт	1	1	2	7500.00	2026-08-29
Скрипка	1	2	3	4500.00	2026-09-07
Виолончель	1	1	1	3000.00	2026-09-02
Контрабас	1	1	1	2500.00	2026-09-08
Балалайка	1	1	1	1000.00	2026-09-10
Классическая гитара	2	2	0	0.00	
Мандолина	1	1	0	0.00	

Групп: 7.

Рисунок 7 — Отчёт приёмщика, сортировка по «paid_rub»

Таблица 6 — Отчёт приёмщика: оплаты картой, фильтр по типу

instrument_type	customers	orders	payments	paid_rub	last_paid_on
Балалайка	1	1	1	1000.00	2026-09-10
Классическая гитара	2	2	0	0.00	
Контрабас	1	1	0	0.00	
Скрипка	1	2	1	2000.00	2026-09-01

Реставратор. Отчёт «Работы по исполнителям и услугам» (листинг 2) соединяет «order_work», «employee», «service» и «restoration_order» и

для каждой пары «исполнитель, услуга» считает работы по статусам и ближайший срок незавершённых работ. В запросе используются только столбцы, разрешённые реставратору: без цены работ, контактов и связи с заказчиком. Параметр «только мои» ограничивает выборку работами текущего сотрудника через функцию «current_employee_id()» из работы № 3. Результат с сортировкой по числу работ показан на рисунке 8, результат с параметром «только мои» — в таблице 7.

```
-- Реставратор: работы по исполнителям и услугам.  
SELECT e.full_name AS restorer, s.name AS service,  
       count(*) AS works,  
       count(*) FILTER (WHERE w.status = 'planned')  
         AS planned,  
       count(*) FILTER (WHERE w.status = 'in_progress')  
         AS in_progress,  
       count(*) FILTER (WHERE w.status = 'done') AS done,  
       min(o.due_on) FILTER (WHERE w.status IN  
                             ('planned', 'in_progress')) AS nearest_due  
FROM order_work w  
JOIN employee e ON e.employee_id = w.employee_id  
JOIN service s ON s.service_id = w.service_id  
JOIN restoration_order o ON o.order_id = w.order_id  
WHERE ((mine)s = ''  
       OR w.employee_id = current_employee_id())  
       AND ((ostatus)s = '' OR o.status = (ostatus)s)  
GROUP BY e.full_name, s.name
```

Листинг 2 — Отчёт реставратора («webapp/sql/workload.sql»)

Работы по исполнителям и услугам

Число работ по статусам и ближайший срок (order_work, employee, service, restoration_order).

Исполнитель: Статус заказа: Фильтр: содержит: Сортировка: по убыванию: Показать

restorer	service	works	planned	in_progress	done	nearest_due
Семенова Алина Петровна	Диагностика	2	0	0	1	
Смирнов Павел Олегович	Замена подставки	1	0	0	1	
Зайцева Дарья Андреевна	Перетяжка струн	1	1	0	0	2026-09-22
Смирнов Павел Олегович	Склейка трещины деки	1	0	1	0	2026-09-20
Козлов Андрей Викторович	Переклейка обечайки	1	0	0	1	
Зайцева Дарья Андреевна	Замена колков	1	0	0	1	
Попова Наталья Юрьевна	Реставрация лака	1	0	0	1	
Волкова Елена Игоревна	Регулировка грифа	1	1	0	0	2026-09-25
Васильев Игорь Сергеевич	Склейка трещины деки	1	1	0	0	2026-10-01
Павлов Роман Евгеньевич	Замена подставки	1	1	0	0	2026-09-20

Групп: 10.

Рисунок 8 — Отчёт реставратора, сортировка по «works»

Таблица 7 — Отчёт реставратора «restorer_smirnov»: только свои работы

restorer	service	works	planned	in_progress	done	nearest_due
Смирнов Павел Олегович	Замена подставки	1	0	0	1	
Смирнов Павел Олегович	Склейка трещины деки	1	0	1	0	2026-09-20

Администратор. Отчёт «Выработка сотрудников» (листинг 3) соединяет «employee», «order_work», «restoration_order» и «instrument» и для каждого сотрудника считает заказы, заказчиков, работы, завершённые работы и согласованную стоимость без отменённых работ. Результат с сортировкой по стоимости приведён в таблице 8. Второй отчёт администратора «Журнал изменений по ролям» (листинг 4) группирует записи журнала по групповой роли, роли входа и таблице; групповая роль определяется по каталогам «pg_roles» и «pg_auth_members» (рисунок 9).

```
-- Администратор: выработка сотрудников.
```

```
SELECT e.full_name AS employee, e.specialization,
       count(DISTINCT w.order_id) AS orders,
       count(DISTINCT i.customer_id) AS customers,
       count(w.line_no) AS works,
```

```

count(*) FILTER (WHERE w.status = 'done') AS done,
round(coalesce(sum(w.agreed_price_kopecks)
    FILTER (WHERE w.status <> 'cancelled'), 0)
    / 100.0, 2) AS agreed_rub
FROM employee e
LEFT JOIN order_work w ON w.employee_id = e.employee_id
LEFT JOIN restoration_order o ON o.order_id = w.order_id
LEFT JOIN instrument i
    ON i.instrument_id = o.instrument_id
WHERE %(ostatus)s = '' OR o.status = %(ostatus)s
GROUP BY e.employee_id, e.full_name, e.specialization

```

Листинг 3 — Отчёт администратора («webapp/sql/employees.sql»)

Таблица 8 — Выработка сотрудников, сортировка по «agreed_rub»

employee	specialization	orders	customers	works	done	agreed_rub
Попова Наталья Юрьевна	Лакокрасочные покрытия	1	1	1	1	7500.00
Смирнов Павел Олегович	Смычковые инструменты	2	1	2	1	7000.00
Козлов Андрей Викторович	Крупные смычковые	1	1	1	0	6000.00
Васильев Игорь Сергеевич	Народные инструменты	1	1	1	0	4500.00
Зайцева Дарья Андреевна	Струны и фурнитура	2	2	2	1	4000.00
Волкова Елена Игоревна	Щипковые инструменты	1	1	1	0	3000.00
Павлов Роман Евгеньевич	Гриффы и подставки	1	1	1	0	2000.00
Семенова Алина Петровна	Диагностика и оценка	2	2	2	1	1000.00

```

-- Администратор: журнал изменений по ролям.
SELECT coalesce(g.rolname, '-') AS app_group,
    l.db_role, l.table_name,
count(*) FILTER (WHERE l.operation = 'I') AS ins,
count(*) FILTER (WHERE l.operation = 'U') AS upd,
count(*) FILTER (WHERE l.operation = 'D') AS del,
count(*) AS total,
to_char(max(l.changed_at), 'YYYY-MM-DD HH24:MI:SS')
    AS last_change
FROM change_log l
LEFT JOIN pg_roles r ON r.rolname = l.db_role
LEFT JOIN pg_auth_members m ON m.member = r.oid

```

```
LEFT JOIN pg_roles g ON g.oid = m.roleid
WHERE %(op)s = ' ' OR l.operation = %(op)s
GROUP BY g.rolname, l.db_role, l.table_name
```

Листинг 4 — Отчёт по журналу («webapp/sql/audit.sql»)

Мастерская · БД workshop
Таблицы
Выработка сотрудников
Журнал изменений по ролям
Пользователи
workshop_admin (администратор)
Выйти

Журнал изменений по ролям

Операции из change_log по группам, ролям входа и таблицам (change_log, pg_roles, pg_auth_members).

Операция: все
Фильтр: — содержит
Сортировка: total по убыванию
Показать

app_group	db_role	table_name	ins	upd	del	total	last_change
admin_role	workshop_admin	employee	0	1	0	1	2026-09-19 12:58:22
restorer_role	restorer_smirnov	inspection	1	0	0	1	2026-09-19 12:58:22
restorer_role	restorer_smirnov	order_work	0	1	0	1	2026-09-19 12:58:22
admin_role	workshop_admin	customer	0	0	1	1	2026-09-19 12:58:22
reception_role	reception_orlova	customer	1	0	0	1	2026-09-19 12:58:22
restorer_role	restorer_kozlov	order_work	0	1	0	1	2026-09-19 12:58:22

Групп: 6.

Рисунок 9 — Отчёт администратора по журналу изменений

3.4 Регистрация пользователей

Страница «Пользователи» администратора (рисунок 10) показывает учётные записи групповых ролей приложения и форму регистрации. При проверке зарегистрированы приёмщик «reception_petrova» и реставратор «restorer_kozlov», связанный с сотрудником № 3; оба вошли в приложение, а «restorer_kozlov» изменил статус своей работы. Попытка зарегистрировать пользователя в «admin_role» отклонена функцией, а прямой вызов функции приёмщиком — ошибкой прав. Пароли новых ролей хранятся в «pg_authid» как SCRAM-верификаторы (таблица 9).

Пользователи приложения

Учётная запись	Групповая роль
workshop_admin	admin_role
reception_orlova	reception_role
reception_petrova	reception_role
restorer_kozlov	restorer_role
restorer_smirnov	restorer_role
restorer_volkova	restorer_role

Регистрация

Пароль преобразуется в SCRAM-верификатор в приложении; СУБД получает только верификатор.

Имя (a-z, 0-9, _)

Роль

приёмщик

Сотрудник (для реставратора)

—

Пароль (не короче 12 символов)

Создать

Рисунок 10 — Пользователи и регистрация

Таблица 9 — Начало хранимых паролей в «pg_authid»

rolname	rolpassword
reception_petrova	SCRAM-SHA-256\$4096:
restorer_kozlov	SCRAM-SHA-256\$4096:
workshop_admin	SCRAM-SHA-256\$4096:

3.5 Контроль журналирования и разграничения доступа

Перед проверкой журнал очищается, поэтому в нём остаются только действия веб-сеансов (таблица 10). Для каждого изменения записана роль, под которой пользователь вошёл через веб-интерфейс, а не общая учётная запись приложения: приёмщик, реставраторы и администратор. Изменение «employee» от имени «workshop_admin» — привязка нового реставратора к сотруднику при регистрации: функция выполняется с правами владельца, но журнал фиксирует роль входа («session_user»). Отклонённые операции в журнал не попали.

Таблица 10 — Журнал изменений после проверки

log_id	db_role	operation	table_name	row_key
1	reception_orlova	I	customer	{'customer_id': 10}

log_id	db_role	operation	table_name	row_key
2	restorer_smirnov	U	order_work	{'line_no': 1, 'order_id': 1}
3	restorer_smirnov	I	inspection	{'inspection_id': 12}
4	workshop_admin	D	customer	{'customer_id': 10}
5	workshop_admin	U	employee	{'employee_id': 3}
6	restorer_kozlov	U	order_work	{'line_no': 1, 'order_id': 4}

Полный вывод проверки приведён в листинге 5: выполнено 49 проверок, все успешны.

```

ok   неверный пароль отклонён [HTTP 401]
ok   postgres (trust) не допускается [HTTP 401]
ok   несуществующий пользователь [HTTP 401]
ok   вход без CSRF-токена [HTTP 400]
ok   блокировка после 5 неудач [HTTP 401, затем 401]
ok   вход reception_orlova [HTTP 302]
ok   вход restorer_smirnov [HTTP 302]
ok   вход workshop_admin [HTTP 302]
ok   заголовки безопасности
ok   cookie HttpOnly, SameSite=Strict
ok   отношений у приёмщика [8]
ok   отношений у реставратора [9]
ok   отношений у администратора [15]
ok   фильтр и сортировка customer [4 строк]
ok   скрыт instrument.customer_id у реставратора
ok   реставратор /t/payment [HTTP 404]
ok   реставратор /t/change_log [HTTP 404]
ok   реставратор /t/customer%3B-- [HTTP 404]
ok   реставратор /r/payments [HTTP 404]
ok   попытка SQL-инъекции в фильтре [0 строк]
ok   приёмщик добавил заказчика
ok   приёмщик не может удалить
      Ошибка СУБД (42501): permission denied for table
      customer
ok   POST с чужим CSRF-токеном [HTTP 400]
ok   реставратор изменил свою работу
ok   чужая работа не изменена (RLS)
ok   цена недоступна для изменения [HTTP 400]
ok   осмотр от чужого имени (RLS)
      Ошибка СУБД (42501): new row violates row-level
      security policy for table "inspection"
ok   осмотр от своего имени

```

ok администратор удалил заказчика
ok удаление связанной строки (FK)
Ошибка СУБД (23503): update or delete on table
"customer" violates foreign key constraint
"instrument_customer_fk" on table "instrument"
ok отчёт payments [7 групп]
ok отчёт workload [10 групп]
ok отчёт employees [8 групп]
ok отчёт audit [4 групп]
ok отчёт payments: фильтр и сортировка [4 групп]
ok отчёт workload: только мои [2 групп]
ok приёмщику отчёт audit недоступен [HTTP 404]
ok регистрация reception_petrova
ok регистрация restorer_kozlov
ok регистрация в admin_role отклонена
Ошибка СУБД (P0001): недопустимая роль: admin_role
ok приёмщику регистрация недоступна [HTTP 403]
ok вход reception_petrova [HTTP 302]
ok вход restorer_kozlov [HTTP 302]
ok restorer_kozlov изменил свою работу
ok функция регистрации закрыта [42501]
ok пароли хранятся как SCRAM
ok журнал: роли входа из веб-сеансов [6 записей]
ok старая cookie после выхода [HTTP 302]
ok истечение сеанса (15 мин) [HTTP 302]

Проверок: 49, успешно: 49

Листинг 5 — Результат «check.py»

4 Защита данных в сервисе

4.1 Реализованные меры

Меры защиты и угрозы, от которых они защищают, сведены в таблицу 11.

Таблица 11 — Угрозы и меры защиты

Угроза	Мера
подбор и перехват пароля	пароль проверяет PostgreSQL по SCRAM-SHA-256; приложение пароли не хранит; блокировка после 5 неудач за 5 минут; одинаковое сообщение об ошибке
вход суперпользователем	отказ, если соединение открыто без пароля или роль — суперпользователь
превышение полномочий	каждый пользователь работает под своей ролью СУБД; права, столбцовые привилегии и политики строк проверяет СУБД; интерфейс лишь скрывает недоступное
подмена сеанса	случайный идентификатор 256 бит в подписанной cookie, данные сеанса на сервере; новый идентификатор при входе; тайм-ауты 15 мин и 8 ч; выход закрывает соединение
CSRF	токен в каждой форме, сравнение за постоянное время; cookie с «SameSite=Strict»
SQL-инъекция	значения — параметрами; имена — из каталога через «sql.Identifier»; параметры отчётов — из списков
XSS и встраивание страницы	автоматическое экранирование Jinja2; Content-Security-Policy «default-src „self“», запрет фреймов; cookie «HttpOnly»
перегрузка СУБД	«statement_timeout» 5 с, не более 200 строк, размер запроса до 64 КиБ
создание привилегированных ролей	регистрация только через функцию с белым списком групп, выполнять её может только администратор; СУБД получает верификатор, а не пароль

Главная особенность решения — разграничение доступа не дублируется в коде. В примере из методических указаний права ролей записаны в словаре приложения, а пароли пользователей хранятся в таблице в открытом виде. Здесь приложение запрашивает у СУБД, что разрешено роли, и строит по этому интерфейс; при изменении привилегий в базе данных код менять не нужно. Даже поддельный запрос, минуя интерфейс (например, удаление от имени приёмщика), отклоняется СУБД, что подтверждено проверкой.

4.2 Аналоги, достоинства и недостатки

Сравнение с распространёнными схемами приведено в таблице 12.

Таблица 12 — Сравнение схем доступа веб-приложения к СУБД

Схема	Достоинства	Недостатки
таблица пользователей и роли в приложении (пример из методических указаний)	простая реализация, общий пул соединений	права дублируются в коде; СУБД и журнал видят одну техническую учётную запись с широкими правами; её компрометация раскрывает всю базу данных
техническая учётная запись и переключение роли СУБД («SET ROLE») по токenu	права проверяет СУБД, пул соединений масштабируется	нужна отдельная служба выдачи токенов; «session_user» в журнале — техническая учётная запись; ошибка в переключении роли даёт её права
соединение под учётной записью пользователя (эта работа)	права и RLS проверяет СУБД; журнал фиксирует настоящего пользователя; приложение не хранит пароли	по соединению на сеанс; сеансы в памяти одного процесса; пароль находится в памяти процесса, пока открыто соединение

Ограничения прототипа:

- встроенный сервер Flask работает по HTTP и предназначен для разработки. Для реальной эксплуатации нужны сервер WSGI за обратным прокси с HTTPS, переменная «IBBD_COOKIE_SECURE=1» и TLS между приложением и СУБД; на стенде всё работает только на адресе 127.0.0.1;
- сеансы и счётчики неудачных попыток хранятся в памяти одного процесса и теряются при перезапуске; несколько процессов их не разделяют;
- блокировка по паре «имя, адрес» позволяет с того же адреса временно заблокировать вход другому пользователю;
- при регистрации пароль ограничен печатными символами ASCII: нормализация SASLprep, которую выполняет PostgreSQL для других символов, в приложении не реализована;
- администратор приложения имеет право удалять строки основных таблиц; удаления фиксируются в журнале, но не требуют подтверждения второго лица.

5 Инструкция по запуску

Необходимы Python 3 (проверено на 3.13.14), PostgreSQL 14 (команды «initdb», «pg_ctl», «psql») для учебного кластера и CLI Typst для сборки отчёта; для снимков экрана — Google Chrome. Команды приведены в листинге 6: создание окружения, пересборка базы данных с проверкой, запуск приложения по адресу `http://127.0.0.1:5000`. Пароли задаются переменными окружения «IBBD_RECEPTION_PASSWORD», «IBBD_RESTORER_PASSWORD» и «IBBD_ADMIN_PASSWORD»; значения по умолчанию предназначены только для демонстрации.

```
cd subjects/ibbd/lab4
make venv      # .venv и пакеты из requirements.txt
make check    # БД ЛР2-ЛР4 и проверка приложения
make run       # http://127.0.0.1:5000
make shots    # снимки экрана (Chrome)
make report    # report.pdf
```

Листинг 6 — Команды запуска

Страница входа с сообщением о неверном пароле показана на рисунке 11.



The screenshot shows a web interface for a 'Мастерская · БД workshop'. At the top, a dark blue header contains the text 'Мастерская · БД workshop'. Below the header, a light pink error message bar displays the text 'Неверное имя пользователя или пароль'. Underneath the error bar, the word 'Вход' (Login) is written in a bold, dark font. Below 'Вход', the text 'Имя и пароль учётной записи PostgreSQL.' is shown. At the bottom, there is a light gray login form with two input fields: 'Пользователь' (Username) containing the text 'reception_orlova' and 'Пароль' (Password) which is empty. To the right of the password field is a button labeled 'Войти' (Login).

Рисунок 11 — Страница входа после неверного пароля

ЗАКЛЮЧЕНИЕ

Разработан многопользовательский веб-интерфейс к базе данных мастерской на Python, Flask и psycorg 3. Пользователь входит под своей учётной записью PostgreSQL, и все запросы выполняются от его имени, поэтому привилегии, столбцовые ограничения, политики строк и журнал из работы № 3 действуют без дублирования в приложении. Список таблиц и столбцов, формы вставки, изменения и удаления строятся по правам роли из каталога СУБД.

Для приёмщика, реставратора и администратора реализованы сводные запросы с группировкой по данным четырёх отношений с параметрами, фильтрацией и сортировкой. Администратор видит все 15 отношений, включая журнал, и регистрирует новых пользователей существующих ролей, передавая СУБД только SCRAM-верификатор пароля.

Автоматическая проверка на настоящей базе данных выполнила 49 проверок входа, сеансов, CSRF, списков таблиц, операций с данными, отчётов, регистрации и журнала; все успешны. Журнал подтвердил, что изменения из веб-интерфейса записываются под ролями настоящих пользователей. Основные ограничения прототипа — работа без HTTPS на встроенном сервере и хранение сеансов в памяти одного процесса.

ПРИЛОЖЕНИЕ А

Скрипт базы данных и модули Python

```
-- Роль администратора приложения и регистрация
-- пользователей. Пароль передаётся переменной psql
-- admin_password; сервер хранит только хэш SCRAM-SHA-256.
BEGIN;

-- 1. Администратор: все таблицы и представления,
--    чтение журнала и шифртекста секретов. Не
--    суперпользователь: RLS и права по-прежнему действуют.
CREATE ROLE admin_role NOLOGIN NOSUPERUSER NOCREATEDB
    NOCREATEROLE NOINHERIT;
GRANT CONNECT ON DATABASE workshop TO admin_role;
GRANT USAGE ON SCHEMA public TO admin_role;
GRANT SELECT, INSERT, UPDATE, DELETE
    ON customer, instrument_type, instrument, employee,
    restoration_order, service, order_work,
    inspection, payment
    TO admin_role;
GRANT SELECT ON v_reception_orders, v_reception_balances,
    v_restorer_tasks, v_restorer inspections
    TO admin_role;
-- Журнал и секреты — только чтение: подделать журнал
-- или заменить шифртекст администратор не может.
GRANT SELECT ON change_log, access_secret TO admin_role;

-- На order_work и inspection включён RLS из ЛРЗ;
-- без политики администратор не увидит ни одной строки.
CREATE POLICY admin_all ON order_work TO admin_role
    USING (true) WITH CHECK (true);
CREATE POLICY admin_all ON inspection TO admin_role
    USING (true) WITH CHECK (true);

-- 2. Регистрация новых пользователей существующих
--    ролей. Приложение передаёт готовый SCRAM-верификатор,
--    поэтому пароль не попадает на сервер в открытом виде.
CREATE FUNCTION app_register_user(
    p_login text, p_group text, p_verifier text,
    p_employee_id integer DEFAULT NULL)
RETURNS void
LANGUAGE plpgsql SECURITY DEFINER
```

```

SET search_path = public, pg_temp
AS $$
BEGIN
    IF p_login !~ '^[a-z][a-z0-9_]{2,39}$'
        OR p_login LIKE 'pg\_%' THEN
        RAISE EXCEPTION 'недопустимое имя: %', p_login;
    END IF;
    IF p_group NOT IN ('reception_role',
        'restorer_role') THEN
        RAISE EXCEPTION 'недопустимая роль: %', p_group;
    END IF;
    IF p_verifier !~ '^SCRAM-SHA-256\$4096:' THEN
        RAISE EXCEPTION 'ожидается SCRAM-верификатор';
    END IF;
    IF EXISTS (SELECT FROM pg_roles
        WHERE rolname = p_login) THEN
        RAISE EXCEPTION 'роль % уже существует', p_login;
    END IF;
    EXECUTE format(
        'CREATE ROLE %I LOGIN INHERIT NOSUPERUSER '
        'NOCREATEDB NOCREATEROLE PASSWORD %L IN ROLE %I',
        p_login, p_verifier, p_group);
    EXECUTE format('COMMENT ON ROLE %I IS %L',
        p_login, 'ibbd-lab4: registered');
    IF p_group = 'restorer_role'
        AND p_employee_id IS NOT NULL THEN
        UPDATE employee SET db_login = p_login
        WHERE employee_id = p_employee_id
        AND db_login IS NULL;
        IF NOT FOUND THEN
            RAISE EXCEPTION 'сотрудник % не найден '
                'или уже связан', p_employee_id;
        END IF;
    END IF;
END;
$$;
REVOKE ALL ON FUNCTION
    app_register_user(text, text, text, integer) FROM PUBLIC;
GRANT EXECUTE ON FUNCTION
    app_register_user(text, text, text, integer)
    TO admin_role;

```

-- 3. Учётная запись администратора.

```
CREATE ROLE workshop_admin LOGIN
    PASSWORD : 'admin_password'
    INHERIT IN ROLE admin_role;
COMMENT ON ROLE workshop_admin IS 'ibbd-lab4: admin';

COMMIT;
```

Листинг А.1 — Администратор и регистрация («sql/01_admin.sql»)

```
"""Вход через СУБД, серверные сессии, CSRF, SCRAM."""
import base64
import hashlib
import hmac
import os
import re
import secrets
import threading
import time
from dataclasses import dataclass, field

import psycopg

# Групповые роли СУБД и соответствующие им виды
# интерфейса; порядок задаёт приоритет.
GROUPS = (
    ("admin_role", "admin"),
    ("reception_role", "reception"),
    ("restorer_role", "restorer"),
)
LOGIN_RE = re.compile(r"^[a-z][a-z0-9_]{0,62}$")
IDLE_TIMEOUT = 15 * 60
ABSOLUTE_TIMEOUT = 8 * 3600
MAX_FAILS = 5
FAIL_WINDOW = 5 * 60
BAD_LOGIN = "Неверное имя пользователя или пароль"

class LoginError(Exception):
    pass

@dataclass
class UserSession:
```

```

conn: psycopg.Connection
login: str
kind: str
created: float = field(default_factory=time.time)
seen: float = field(default_factory=time.time)
lock: threading.Lock = field(
    default_factory=threading.Lock)

def expired(self, now):
    return (now - self.seen > IDLE_TIMEOUT
            or now - self.created > ABSOLUTE_TIMEOUT)

class SessionStore:
    """Сессии и неудачные попытки хранятся в памяти
    процесса; в cookie попадает только случайный sid."""

    def __init__(self):
        self._lock = threading.Lock()
        self._items = {}
        self._fails = {}

    def add(self, us):
        sid = secrets.token_urlsafe(32)
        with self._lock:
            self._items[sid] = us
        return sid

    def get(self, sid):
        if not sid:
            return None
        now = time.time()
        with self._lock:
            us = self._items.get(sid)
            if us is None:
                return None
            if us.expired(now) or us.conn.closed:
                del self._items[sid]
            else:
                us.seen = now
                return us
        us.conn.close()
        return None

```

```

def drop(self, sid):
    with self._lock:
        us = self._items.pop(sid, None)
        if us is not None:
            us.conn.close()

def locked(self, key):
    now = time.time()
    with self._lock:
        recent = [t for t in self._fails.get(key, ())
                    if now - t < FAIL_WINDOW]
        self._fails[key] = recent
        return len(recent) >= MAX_FAILS

def fail(self, key):
    with self._lock:
        self._fails.setdefault(key, []).append(
            time.time())

def reset(self, key):
    with self._lock:
        self._fails.pop(key, None)

def login(store, cfg, username, password, addr):
    """Открывает соединение с СУБД от имени пользователя.
    Пароль проверяет PostgreSQL (SCRAM-SHA-256)."""
    key = (username, addr)
    if store.locked(key):
        raise LoginError("Слишком много неудачных попыток; "
                          "повторите вход позже")
    if not LOGIN_RE.match(username) or not password:
        store.fail(key)
        raise LoginError(BAD_LOGIN)
    try:
        conn = psycopg2.connect(
            host=cfg["PGHOST"], port=cfg["PGPORT"],
            dbname=cfg["PGDATABASE"], user=username,
            password=password, connect_timeout=5,
            application_name="ibbd-lab4", autocommit=True)
    except psycopg2.OperationalError:
        store.fail(key)

```

```

        raise LoginError(BAD_LOGIN) from None
    try:
        kind = check_account(conn)
    except Exception:
        conn.close()
        raise
    if kind is None:
        conn.close()
        store.fail(key)
        raise LoginError(BAD_LOGIN)
    conn.execute("SET statement_timeout = '5s'")
    store.reset(key)
    return store.add(UserSession(conn, username, kind))

def check_account(conn):
    """Вид интерфейса или None, если вход не допускается."""
    # Вход без пароля (trust) и суперпользователь
    # в веб-приложение не допускаются.
    if not conn.pgconn.used_password:
        return None
    row = conn.execute(
        "SELECT rolsuper FROM pg_roles"
        " WHERE rolname = current_user").fetchone()
    if row is None or row[0]:
        return None
    for group, kind in GROUPS:
        member = conn.execute(
            "SELECT pg_has_role(current_user, %s, 'MEMBER')",
            (group,)).fetchone()[0]
        if member:
            return kind
    return None

def new_csrf():
    return secrets.token_urlsafe(32)

def csrf_ok(expected, got):
    return bool(expected) and bool(got) and \
        hmac.compare_digest(expected, got)

```

```

def scram_verifier(password, iterations=4096):
    """SCRAM-SHA-256-верификатор в формате PostgreSQL
    (RFC 5802, RFC 7677): открытый пароль не
    передаётся серверу."""
    salt = os.urandom(16)
    salted = hashlib.pbkdf2_hmac(
        "sha256", password.encode(), salt, iterations)
    client = hmac.new(salted, b"Client Key",
        "sha256").digest()
    stored = hashlib.sha256(client).digest()
    server = hmac.new(salted, b"Server Key",
        "sha256").digest()

    def b64(b):
        return base64.b64encode(b).decode()

    return (f"SCRAM-SHA-256${iterations}:{b64(salt)}"
        f"${b64(stored)}:{b64(server)}")

```

Листинг А.2 — Аутентификация («webapp/auth.py»)

```

"""Каталог доступных отношений и операции с таблицами.
Имена отношений и столбцов берутся только из каталога
PostgreSQL для текущей роли и подставляются через
psycopg.sql.Identifier; значения — только параметрами."""
from dataclasses import dataclass

from psycopg import sql

ROW_LIMIT = 200

RELATIONS_SQL = """
SELECT c.relname, c.relkind = 'v',
       has_any_column_privilege(c.oid, 'INSERT'),
       has_any_column_privilege(c.oid, 'UPDATE'),
       has_table_privilege(c.oid, 'DELETE'),
       has_table_privilege(c.oid, 'SELECT')
FROM pg_class c
WHERE c.relnamespace = 'public'::regnamespace
      AND c.relkind IN ('r', 'v')
      AND has_any_column_privilege(c.oid, 'SELECT')
ORDER BY c.relkind, c.relname

```

```
"""
```

```
COLUMNS_SQL = """
SELECT a.attname, format_type(a.atttypid, a.atttypmod),
       has_column_privilege(c.oid, a.attnum, 'SELECT'),
       has_column_privilege(c.oid, a.attnum, 'INSERT'),
       has_column_privilege(c.oid, a.attnum, 'UPDATE'),
       a.attidentity <> '' OR a.atthasdef,
       coalesce(a.attnum = ANY (i.indkey), false)
FROM pg_class c
JOIN pg_attribute a ON a.attrelid = c.oid
LEFT JOIN pg_index i
      ON i.indrelid = c.oid AND i.indisprimary
WHERE c.relnamespace = 'public'::regnamespace
      AND c.relname = %s
      AND a.attnum > 0 AND NOT a.attisdropped
ORDER BY a.attnum
"""
```

```
@dataclass
class Relation:
    name: str
    is_view: bool
    can_insert: bool
    can_update: bool
    can_delete: bool
    full_select: bool
```

```
@dataclass
class Column:
    name: str
    type: str
    can_select: bool
    can_insert: bool
    can_update: bool
    has_default: bool
    is_pk: bool
```

```
def relations(conn):
    """Отношения, на которые у роли есть право SELECT
```

```

        хотя бы на один столбец. """
        rows = conn.execute(RELATIONS_SQL).fetchall()
        return [Relation(*r) for r in rows]

def find_relation(conn, name):
    for rel in relations(conn):
        if rel.name == name:
            return rel
    return None

def columns(conn, rel):
    rows = conn.execute(COLUMNS_SQL, (rel,)).fetchall()
    return [Column(*r) for r in rows]

def where_filter(col, op, value):
    """Условие фильтра: столбец из белого списка,
    значение – параметр."""
    ident = sql.Identifier(col)
    if op == "eq":
        return (sql.SQL("{}::text = %(fval)s").format(ident),
                {"fval": value})
    pattern = "%" + value.replace("\\", "\\") \
        .replace("%", "\\%").replace("_", "\\_") + "%"
    return (sql.SQL("{}::text ILIKE %(fval)s")
            .format(ident), {"fval": pattern})

def order_by(col, desc):
    direction = sql.SQL("DESC" if desc else "ASC")
    return sql.SQL(" ORDER BY {} {} NULLS LAST").format(
        sql.Identifier(col), direction)

def select_rows(conn, rel, cols, flt=None, sort=None):
    """flt = (столбец, операция, значение),
    sort = (столбец, по убыванию)."""
    query = sql.SQL("SELECT {} FROM {}").format(
        sql.SQL(", ").join(map(sql.Identifier, cols)),
        sql.Identifier(rel))
    params = {"limit": ROW_LIMIT}

```

```

if flt:
    cond, p = where_filter(*flt)
    query += sql.SQL(" WHERE ") + cond
    params.update(p)
if sort:
    query += order_by(*sort)
query += sql.SQL(" LIMIT %(limit)s")
return conn.execute(query, params).fetchall()

def key_condition(keys):
    return sql.SQL(" AND ").join(
        sql.SQL("{} = %s").format(sql.Identifier(k))
        for k in keys)

def insert_row(conn, rel, values):
    """values: {столбец: строка}; пустые поля не
    передаются, и СУБД подставляет значения по
    умолчанию."""
    names = list(values)
    query = sql.SQL("INSERT INTO {} ({} ) VALUES ({} )").format(
        sql.Identifier(rel),
        sql.SQL(", ").join(map(sql.Identifier, names)),
        sql.SQL(", ").join(sql.Placeholder() * len(names)))
    return conn.execute(query, list(values.values())).rowcount

def delete_row(conn, rel, key):
    query = sql.SQL("DELETE FROM {} WHERE ").format(
        sql.Identifier(rel)) + key_condition(key)
    return conn.execute(query, list(key.values())).rowcount

def update_value(conn, rel, key, col, value):
    query = sql.SQL("UPDATE {} SET {} = %s WHERE ").format(
        sql.Identifier(rel), sql.Identifier(col)) \
        + key_condition(key)
    params = [value] + list(key.values())
    return conn.execute(query, params).rowcount

```

Листинг А.3 — Доступ к данным («webapp/db.py»)

```
"""Сводные запросы с GROUP BY для каждой группы
пользователей. Параметры отчёта принимают значения
только из заданных списков."""
```

```
from dataclasses import dataclass
```

```
from pathlib import Path
```

```
from psycopg import sql
```

```
from .db import order_by, where_filter
```

```
@dataclass
```

```
class Param:
```

```
    name: str
```

```
    label: str
```

```
    choices: tuple # (значение, подпись); "" — все
```

```
@dataclass
```

```
class Report:
```

```
    key: str
```

```
    kind: str
```

```
    title: str
```

```
    about: str
```

```
    query: str
```

```
    params: tuple = ()
```

```
SQL_DIR = Path(__file__).parent / "sql"
```

```
def load(key):
```

```
    return (SQL_DIR / f"{key}.sql").read_text("utf-8")
```

```
ORDER_STATUS = Param("ostatus", "Статус заказа", (
    ("", "любой"), ("accepted", "accepted"),
    ("in_progress", "in_progress"), ("ready", "ready"),
    ("closed", "closed"), ("cancelled", "cancelled")))

```

```
REPORTS = (
```

```
    Report("payments", "reception",
```

```
        "Оплаты по типам инструментов",
```

```

        "Заказчики, заказы и поступившие оплаты по "
        "типам инструментов (instrument_type, "
        "instrument, restoration_order, payment).",
        load("payments"),
        (ORDER_STATUS,
         Param("method", "Способ оплаты", (
             ("", "любой"), ("cash", "cash"),
             ("card", "card"),
             ("transfer", "transfer"))))),
    Report("workload", "restorer",
          "Работы по исполнителям и услугам",
          "Число работ по статусам и ближайший срок "
          "(order_work, employee, service, "
          "restoration_order).",
          load("workload"),
          (Param("mine", "Исполнитель", (
              ("", "все"), ("1", "только мои"))),
           ORDER_STATUS)),
    Report("employees", "admin",
          "Выработка сотрудников",
          "Заказы, заказчики, работы и согласованная "
          "стоимость по сотрудникам (employee, "
          "order_work, restoration_order, instrument).",
          load("employees"), (ORDER_STATUS,)),
    Report("audit", "admin",
          "Журнал изменений по ролям",
          "Операции из change_log по группам, ролям "
          "входа и таблицам (change_log, pg_roles, "
          "pg_auth_members).",
          load("audit"),
          (Param("op", "Операция", (
              ("", "все"), ("I", "INSERT"),
              ("U", "UPDATE"), ("D", "DELETE")))),),
    )

```

```

def for_kind(kind):
    return [r for r in REPORTS if r.kind == kind]

```

```

def find(kind, key):
    for r in for_kind(kind):
        if r.key == key:

```

```

        return r
    return None

def clean_params(report, args):
    """Значение параметра вне списка заменяется на ""."""
    values = {}
    for p in report.params:
        v = args.get(p.name, "")
        allowed = {c[0] for c in p.choices}
        values[p.name] = v if v in allowed else ""
    return values

def columns(conn, report, params):
    """Имена столбцов результата – белый список для
    сортировки и фильтра."""
    query = sql.SQL("SELECT * FROM ({}) r LIMIT 0").format(
        sql.SQL(report.query))
    cur = conn.execute(query, params)
    return [d.name for d in cur.description]

def run(conn, report, params, flt=None, sort=None):
    query = sql.SQL("SELECT * FROM ({}) r").format(
        sql.SQL(report.query))
    params = dict(params)
    if flt:
        cond, p = where_filter(*flt)
        query += sql.SQL(" WHERE ") + cond
        params.update(p)
    if sort:
        query += order_by(*sort)
    return conn.execute(query, params).fetchall()

```

Листинг А.4 – Отчёты («webapp/reports.py»)

ПРИЛОЖЕНИЕ Б

Приложение Flask

```
"""Веб-интерфейс БД workshop: маршруты Flask."""
import functools
import os
import re
import secrets

import psycpg
from flask import (Flask, abort, flash, g, redirect,
                   render_template, request, session,
                   url_for)

from . import auth, db, reports

KIND_TITLE = {"admin": "администратор",
              "reception": "приёмщик",
              "restorer": "реставратор"}
PASSWORD_RE = re.compile(r"^\[\x21-\x7e\]{12,64}$")

def create_app(test_config=None):
    app = Flask(__name__)
    app.config.update(
        SECRET_KEY=os.environ.get("IBBD_SECRET_KEY")
        or secrets.token_bytes(32),
        SESSION_COOKIE_HTTPONLY=True,
        SESSION_COOKIE_SAMESITE="Strict",
        SESSION_COOKIE_SECURE=os.environ.get(
            "IBBD_COOKIE_SECURE") == "1",
        MAX_CONTENT_LENGTH=64 * 1024,
        PGHOST=os.environ.get("IBBD_PGHOST", "127.0.0.1"),
        PGPORT=int(os.environ.get("IBBD_PGPORT", "55432")),
        PGDATABASE=os.environ.get("IBBD_DB", "workshop"),
    )
    if test_config:
        app.config.update(test_config)
    store = auth.SessionStore()
    app.extensions["ibbd_store"] = store

    @app.before_request
```

```

def load_user():
    g.us = store.get(session.get("sid"))
    if session.get("sid") and g.us is None:
        session.pop("sid")
    if "csrf" not in session:
        session["csrf"] = auth.new_csrf()
    if request.method == "POST" and not auth.csrf_ok(
        session["csrf"], request.form.get("csrf")):
        abort(400, "Неверный CSRF-токен")

@app.after_request
def headers(resp):
    h = resp.headers
    h["Content-Security-Policy"] = (
        "default-src 'self'; frame-ancestors 'none'")
    h["X-Content-Type-Options"] = "nosniff"
    h["X-Frame-Options"] = "DENY"
    h["Referrer-Policy"] = "no-referrer"
    h["Cache-Control"] = "no-store"
    return resp

@app.context_processor
def common():
    us = g.get("us")
    return {"us": us, "csrf": session.get("csrf"),
            "kind_title": KIND_TITLE,
            "reports": reports.for_kind(us.kind)
            if us else []}

def login_required(view):
    @functools.wraps(view)
    def wrapper(*args, **kwargs):
        if g.us is None:
            return redirect(url_for("login_form"))
        with g.us.lock:
            try:
                return view(*args, **kwargs)
            except psycopg.OperationalError:
                store.drop(session.pop("sid", None))
                flash("Соединение с СУБД потеряно")
                return redirect(url_for("login_form"))
    return wrapper

```

```

def db_error(e):
    msg = e.diag.message_primary or str(e)
    flash(f"Ошибка СУБД ({e.sqlstate}): {msg}", "error")

@app.get("/")
def index():
    if g.us is None:
        return redirect(url_for("login_form"))
    return redirect(url_for("home"))

@app.get("/login")
def login_form():
    return render_template("login.html")

@app.post("/login")
def login():
    name = request.form.get("username", "").strip()
    password = request.form.get("password", "")
    try:
        sid = auth.login(store, app.config, name,
                        password, request.remote_addr)
    except auth.LoginError as e:
        flash(str(e), "error")
        return render_template("login.html",
                               username=name), 401

    # Новая сессия и новый CSRF-токен после входа.
    store.drop(session.get("sid"))
    session.clear()
    session["sid"] = sid
    session["csrf"] = auth.new_csrf()
    return redirect(url_for("home"))

@app.post("/logout")
def logout():
    store.drop(session.pop("sid", None))
    session.clear()
    flash("Сеанс завершён")
    return redirect(url_for("login_form"))

@app.get("/tables")
@login_required
def home():
    return render_template(

```

```

        "home.html", relations=db.relations(g.us.conn))

def table_context(name):
    rel = db.find_relation(g.us.conn, name)
    if rel is None:
        abort(404)
    cols = db.columns(g.us.conn, name)
    visible = [c for c in cols if c.can_select]
    return rel, cols, visible

def filter_sort(names):
    fcol = request.args.get("fcol", "")
    fop = request.args.get("fop", "like")
    fval = request.args.get("fval", "")
    sort = request.args.get("sort", "")
    desc = request.args.get("dir") == "desc"
    flt = None
    if fcol in names and fval:
        flt = (fcol, "eq" if fop == "eq" else "like",
              fval)
    srt = (sort, desc) if sort in names else None
    return flt, srt

@app.get("/t/<name>")
@login_required
def table(name):
    rel, cols, visible = table_context(name)
    names = [c.name for c in visible]
    flt, srt = filter_sort(names)
    rows = []
    try:
        rows = db.select_rows(g.us.conn, name, names,
                              flt, srt)
    except psycopg.Error as e:
        db_error(e)
    keys = [c.name for c in cols if c.is_pk]
    deletable = (rel.can_delete and keys
                 and set(keys) <= set(names))
    return render_template(
        "table.html", rel=rel, cols=cols,
        visible=visible, rows=rows, keys=keys,
        key_idx=[names.index(k) for k in keys]
        if deletable else [],

```

```

        deletable=deletable, args=request.args,
        limit=db.ROW_LIMIT)

def key_values(cols):
    keys = [c.name for c in cols if c.is_pk]
    return {k: request.form.get("key_" + k, "")
            for k in keys}

@app.post("/t/<name>/insert")
@login_required
def insert(name):
    rel, cols, _ = table_context(name)
    values = {c.name: request.form.get("col_" + c.name)
              for c in cols if c.can_insert}
    values = {k: v for k, v in values.items() if v}
    if not values:
        flash("Не заполнено ни одного поля", "error")
    else:
        try:
            n = db.insert_row(g.us.conn, name, values)
            flash(f"Добавлено строк: {n}")
        except psycopg.Error as e:
            db_error(e)
    return redirect(url_for("table", name=name))

@app.post("/t/<name>/delete")
@login_required
def delete(name):
    _, cols, _ = table_context(name)
    key = key_values(cols)
    if not key or not all(key.values()):
        abort(400)
    try:
        n = db.delete_row(g.us.conn, name, key)
        flash(f"Удалено строк: {n}")
    except psycopg.Error as e:
        db_error(e)
    return redirect(url_for("table", name=name))

@app.post("/t/<name>/update")
@login_required
def update(name):
    _, cols, _ = table_context(name)

```

```

key = key_values(cols)
col = request.form.get("column", "")
allowed = {c.name for c in cols if c.can_update}
if not key or not all(key.values()) \
    or col not in allowed:
    abort(400)
value = request.form.get("value") or None
try:
    n = db.update_value(g.us.conn, name, key, col,
                        value)
    flash(f"Изменено строка: {n}")
except psycopg.Error as e:
    db_error(e)
return redirect(url_for("table", name=name))

@app.get("/r/<key>")
@login_required
def report(key):
    rep = reports.find(g.us.kind, key)
    if rep is None:
        abort(404)
    params = reports.clean_params(rep, request.args)
    names, rows = [], []
    try:
        names = reports.columns(g.us.conn, rep, params)
        flt, srt = filter_sort(names)
        rows = reports.run(g.us.conn, rep, params,
                           flt, srt)
    except psycopg.Error as e:
        db_error(e)
    return render_template(
        "report.html", rep=rep, names=names, rows=rows,
        params=params, args=request.args)

@app.route("/register", methods=["GET", "POST"])
@login_required
def register():
    if g.us.kind != "admin":
        abort(403)
    conn = g.us.conn
    if request.method == "POST":
        login_ = request.form.get("login", "")
        group = request.form.get("group", "")

```

```

pw = request.form.get("password", "")
emp = request.form.get("employee_id") or None
if not PASSWORD_RE.match(pw):
    flash("Пароль: 12–64 печатных символа "
          "ASCII без пробелов", "error")
else:
    try:
        conn.execute(
            "SELECT app_register_user("
            "%s, %s, %s, %s)",
            (login_, group,
             auth.scram_verifier(pw), emp))
        flash(f"Создан пользователь {login_}")
        return redirect(url_for("register"))
    except psycopg2.Error as e:
        db_error(e)
users = conn.execute(
    "SELECT r.rolname, g.rolname"
    " FROM pg_roles r"
    " JOIN pg_auth_members m ON m.member = r.oid"
    " JOIN pg_roles g ON g.oid = m.roleid"
    " WHERE g.rolname IN ('reception_role',"
    " 'restorer_role', 'admin_role')"
    " ORDER BY 2, 1").fetchall()
free = conn.execute(
    "SELECT employee_id, full_name FROM employee"
    " WHERE db_login IS NULL"
    " ORDER BY employee_id").fetchall()
return render_template("register.html",
                       users=users, free=free)

return app

```

Листинг Б.1 — Маршруты («webapp/__init__.py»)

```

"""Запуск: python -m webapp (только 127.0.0.1)."""
import os

from . import create_app

if __name__ == "__main__":
    create_app().run(
        host="127.0.0.1",

```

```
port=int(os.environ.get("IBBD_WEB_PORT", "5000")),
debug=False)
```

Листинг Б.2 — Запуск («webapp/__main__.py»)

```
Flask==3.1.3
psycorg==3.3.6
# только для make shots (использует установленный Chrome)
playwright==1.63.0
```

Листинг Б.3 — Зависимости («requirements.txt»)

```
TYPST ?= typst
```

```
PY := .venv/bin/python
```

```
.PHONY: help venv check shots run report
```

```
help:
```

```
@printf '%s\n' \
    'make venv    - create .venv and install requirements' \
    'make check  - rebuild DB, test app, save verification' \
    'make shots  - screenshots of the running app (Chrome)' \
    'make run    - start app on http://127.0.0.1:5000' \
    'make report - build report.pdf'
```

```
venv:
```

```
python3 -m venv .venv
$(PY) -m pip install -r requirements.txt
```

```
check:
```

```
./check.sh
```

```
shots:
```

```
$(PY) shots.py shots
```

```
run:
```

```
$(PY) -m webapp
```

```
report:
```

```
$(TYPST) compile --root ../../.. report.typ report.pdf
```

Листинг Б.4 — Команды («Makefile»)

ПРИЛОЖЕНИЕ В

Шаблоны и стили

Шаблоны используют синтаксис Jinja2: общий шаблон (листинг В.1), вход (листинг В.2), список отношений (листинг В.3), форма фильтра (листинг В.4), таблица с формами (листинг В.5), отчёт (листинг В.6), регистрация (листинг В.7) и стили (листинг В.8).

```
<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<title>{% block title %}{% endblock %} · workshop</title>
<link rel="stylesheet"
      href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
<header>
  <b>Мастерская · БД workshop</b>
  {% if us %}
  <nav>
    <a href="{{ url_for('home') }}">Таблицы</a>
    {% for r in reports %}
    <a href="{{ url_for('report', key=r.key) }}">
      {{ r.title }}</a>
    {% endfor %}
    {% if us.kind == 'admin' %}
    <a href="{{ url_for('register') }}">Пользователи</a>
    {% endif %}
  </nav>
  <form method="post" action="{{ url_for('logout') }}">
    <input type="hidden" name="csrf" value="{{ csrf }}">
    {{ us.login }} ({{ kind_title[us.kind] }})
    <button>Выйти</button>
  </form>
  {% endif %}
</header>
<main>
{% for cat, msg in get_flashed_messages(true) %}
  <p class="flash {{ cat }}">{{ msg }}</p>
{% endfor %}
{% block body %}{% endblock %}
```

```

</main>
</body>
</html>

```

Листинг В.1 — Общий шаблон («base.html»)

```

{% extends "base.html" %}
{% block title %}Вход{% endblock %}
{% block body %}
<h1>Вход</h1>
<p>Имя и пароль учётной записи PostgreSQL.</p>
<form method="post" action="{{ url_for('login') }}"
      class="card">
  <input type="hidden" name="csrf" value="{{ csrf }}">
  <label>Пользователь
    <input name="username" value="{{ username }}"
          autocomplete="username" required></label>
  <label>Пароль
    <input name="password" type="password"
          autocomplete="current-password" required></label>
  <button>Войти</button>
</form>
{% endblock %}

```

Листинг В.2 — Вход («login.html»)

```

{% extends "base.html" %}
{% block title %}Таблицы{% endblock %}
{% block body %}
<h1>Доступные отношения</h1>
<p>Список построен по привилегиям роли {{ us.login }}
в СУБД. S* – чтение части столбцов.</p>
<table>
<tr><th>Отношение</th><th>Вид</th><th>Чтение</th>
    <th>Вставка</th><th>Изменение</th><th>Удаление</th></tr>
{% for r in relations %}
<tr>
  <td><a href="{{ url_for('table', name=r.name) }}">
    {{ r.name }}</a></td>
  <td>{{ 'VIEW' if r.is_view else 'TABLE' }}</td>
  <td>{{ 'S' if r.full_select else 'S*' }}</td>
  <td>{{ 'I' if r.can_insert }}</td>
  <td>{{ 'U' if r.can_update }}</td>
  <td>{{ 'D' if r.can_delete }}</td>

```

```

</tr>
{% endfor %}
</table>
<h2>Отчёты</h2>
<ul>
{% for r in reports %}
  <li><a href="{{ url_for('report', key=r.key) }}">
    {{ r.title }}</a> – {{ r.about }}</li>
{% endfor %}
</ul>
{% endblock %}

```

Листинг В.3 — Список отношений («home.html»)

```

{# Общая форма фильтра и сортировки. #}
<form method="get" class="bar">
  {% for p in extra %}
    <label>{{ p.label }}
    <select name="{{ p.name }}">
      {% for v, t in p.choices %}
        <option value="{{ v }}"
          {{ 'selected' if params[p.name] == v }}>{{ t }}
        </option>
      {% endfor %}
    </select></label>
  {% endfor %}
  <label>Фильтр
    <select name="fcol">
      <option value=""></option>
      {% for n in names %}
        <option {{ 'selected' if args.fcol == n }}>
          {{- n -}}</option>
      {% endfor %}
    </select></label>
    <select name="fop">
      <option value="like">содержит</option>
      <option value="eq" {{ 'selected'
        if args.fop == 'eq' }}>равно</option>
    </select>
    <input name="fval" value="{{ args.fval }}">
    <label>Сортировка
      <select name="sort">
        <option value=""></option>

```

```

        {% for n in names %}
        <option {{ 'selected' if args.sort == n }}>
            {{- n -}}</option>
        {% endfor %}
    </select></label>
<select name="dir">
    <option value="asc">по возрастанию</option>
    <option value="desc" {{ 'selected'
        if args.dir == 'desc' }}>по убыванию</option>
</select>
<button>Показать</button>
</form>

```

Листинг В.4 — Фильтр и сортировка («_filter.html»)

```

{% extends "base.html" %}
{% block title %}{{ rel.name }}{% endblock %}
{% block body %}
<h1>{{ rel.name }}</h1>
{% set names = visible | map(attribute='name') | list %}
{% set extra = [] %}
{% set params = {} %}
{% include "_filter.html" %}
<p>Строк: {{ rows | length }} (не более {{ limit }}).
{% if cols | length > visible | length %}
Скрыты столбцы без права чтения:
{{ cols | rejectattr('can_select')
    | map(attribute='name') | join(', ') }}.
{% endif %}</p>
<div class="scroll"><table>
<tr>{% for c in visible %}<th>{{ c.name }}</th>{% endfor %}
    {% if deletable %}<th></th>{% endif %}</tr>
{% for row in rows %}
<tr>{% for v in row %}<td>{{ '' if v is none else v }}</td>
    {% endfor %}
    {% if deletable %}<td>
<form method="post"
    action="{{ url_for('delete', name=rel.name) }}">
    <input type="hidden" name="csrf" value="{{ csrf }}">
    {% for i in key_idx %}
    <input type="hidden" name="key_{{ keys[loop.index0] }}"
        value="{{ row[i] }}">
    {% endfor %}

```

```

        <button class="danger">Удалить</button>
    </form></td>{% endif %}
</tr>
{% endfor %}
</table></div>

{% if rel.can_insert %}
<h2>Добавить строку</h2>
<form method="post" class="card"
    action="{{ url_for('insert', name=rel.name) }}">
    <input type="hidden" name="csrf" value="{{ csrf }}">
    {% for c in cols if c.can_insert %}
    <label>{{ c.name }} <small>{{ c.type }}
        {{- ' , по умолчанию' if c.has_default }}</small>
        <input name="col_{{ c.name }}"></label>
    {% endfor %}
    <button>Добавить</button>
</form>
{% endif %}

{% if rel.can_update and keys %}
<h2>Изменить значение</h2>
<form method="post" class="card"
    action="{{ url_for('update', name=rel.name) }}">
    <input type="hidden" name="csrf" value="{{ csrf }}">
    {% for k in keys %}
    <label>{{ k }} (ключ)
        <input name="key_{{ k }}" required></label>
    {% endfor %}
    <label>Столбец
        <select name="column">
            {% for c in cols if c.can_update %}
            <option>{{ c.name }}</option>
            {% endfor %}
        </select></label>
    <label>Новое значение <input name="value"></label>
    <button>Изменить</button>
</form>
{% endif %}
{% endblock %}

```

Листинг В.5 — Таблица и формы («table.html»)

```

{% extends "base.html" %}
{% block title %}{{ rep.title }}{% endblock %}
{% block body %}
<h1>{{ rep.title }}</h1>
<p>{{ rep.about }}</p>
{% set extra = rep.params %}
{% include "_filter.html" %}
<div class="scroll"><table>
<tr>{% for n in names %}<th>{{ n }}</th>{% endfor %}</tr>
{% for row in rows %}
<tr>{% for v in row %}<td>{{ ' ' if v is none else v }}</td>
    {% endfor %}</tr>
{% endfor %}
</table></div>
<p>Группы: {{ rows | length }}.</p>
{% endblock %}

```

Листинг В.6 — Отчёт («report.html»)

```

{% extends "base.html" %}
{% block title %}Пользователи{% endblock %}
{% block body %}
<h1>Пользователи приложения</h1>
<table>
<tr><th>Учётная запись</th><th>Групповая роль</th></tr>
{% for u, grp in users %}
<tr><td>{{ u }}</td><td>{{ grp }}</td></tr>
{% endfor %}
</table>
<h2>Регистрация</h2>
<p>Пароль преобразуется в SCRAM-верификатор в
приложении; СУБД получает только верификатор.</p>
<form method="post" class="card">
    <input type="hidden" name="csrf" value="{{ csrf }}">
    <label>Имя (a-z, 0-9, _)<br>
        <input name="login" required
            pattern="[a-z][a-z0-9_]{2,39}"></label>
    <label>Роль<br>
        <select name="group">
            <option value="reception_role">приёмщик</option>
            <option value="restorer_role">реставратор</option>
        </select></label>
    <label>Сотрудник (для реставратора)

```

```

<select name="employee_id">
  <option value="">—</option>
  {% for id, name in free %}
    <option value="{{ id }}">{{ id }}. {{ name }}
  </option>
  {% endfor %}
</select></label>
<label>Пароль (не короче 12 символов)
  <input name="password" type="password" required
    autocomplete="new-password"></label>
<button>Создать</button>
</form>
{% endblock %}

```

Листинг В.7 — Регистрация («register.html»)

```

body { font: 15px/1.4 system-ui, sans-serif;
       margin: 0; color: #222; }
header { display: flex; gap: 1.5em; align-items: center;
        flex-wrap: wrap; padding: .6em 1.2em;
        background: #2d4a6b; color: #fff; }
header a { color: #fff; margin-right: .8em; }
header form { margin-left: auto; }
main { padding: 1em 1.2em; }
table { border-collapse: collapse; margin: .5em 0; }
th, td { border: 1px solid #bbb; padding: .25em .5em;
        text-align: left; vertical-align: top; }
th { background: #e8eef5; }
.scroll { overflow-x: auto; }
.card { display: flex; flex-wrap: wrap; gap: .6em 1em;
       align-items: flex-end; padding: .8em;
       background: #f5f5f5; max-width: 60em; }
.card label, .bar label { display: flex;
                        flex-direction: column; font-size: 13px; }
.bar { display: flex; flex-wrap: wrap; gap: .5em;
      align-items: flex-end; margin: .5em 0; }
.flash { padding: .5em .8em; background: #e3f3e3; }
.flash.error { background: #fbe3e3; }
button.danger { color: #a00; }

```

Листинг В.8 — Стили («style.css»)

ПРИЛОЖЕНИЕ Г

Сценарии проверки

Сценарий «check.sh» (листинг Г.1) пересобирает базу данных сценарием работы № 3, удаляет роли прошлого запуска, применяет скрипт этой работы, очищает журнал и запускает «check.py» (листинг Г.2). Сценарий «shots.py» (листинг Г.3) запускает приложение и получает снимки экрана.

```
#!/bin/sh
# Пересобирает БД ЛР2–ЛР3, добавляет администратора и
# регистрацию ЛР4, проверяет веб-приложение и сохраняет
# выводы в verification/.
set -eu

LAB_DIR=$(cd "$(dirname "$0")" && pwd)
PG="$LAB_DIR/../tools/pg.sh"
OUT="${IBBD_OUT:-$LAB_DIR/verification}"
PY="$LAB_DIR/.venv/bin/python"
ADMIN_PASSWORD="${IBBD_ADMIN_PASSWORD:-admin-demo-pass}"

[ -x "$PY" ] || { echo "нет .venv: make venv" >&2; exit 1; }

# БД ЛР3 с нуля; её выводы не нужны.
LAB3_OUT=$(mktemp -d)
IBBD_OUT="$LAB3_OUT" "$LAB_DIR/../lab3/check.sh" >/dev/null
rm -rf "$LAB3_OUT"

# Роли ЛР4 (в том числе зарегистрированные) общие для
# кластера: удалить оставшиеся от прошлого запуска.
"$PG" psql -d postgres -q -v ON_ERROR_STOP=1 <<'SQL'
DO $$
DECLARE r record;
BEGIN
    FOR r IN SELECT rolname FROM pg_roles
              WHERE shobj_description(oid, 'pg_authid')
              LIKE 'ibbd-lab4:%'
    LOOP
        EXECUTE format('DROP ROLE %I', r.rolname);
    END LOOP;
END $$;
DROP ROLE IF EXISTS admin_role;
```

SQL

```
"$PG" psql -d workshop -q -v ON_ERROR_STOP=1 \  
-v admin_password="$ADMIN_PASSWORD" \  
-f "$LAB_DIR/sql/01_admin.sql" >/dev/null  
  
# В журнале остаются только действия веб-сеансов.  
"$PG" psql -d workshop -q -c \  
"TRUNCATE change_log RESTART IDENTITY" >/dev/null  
  
mkdir -p "$OUT"  
cd "$LAB_DIR"  
"$PY" check.py "$OUT"
```

Листинг Г.1 — Пересборка и проверка («check.sh»)

```
"""Проверка веб-приложения на настоящей БД workshop.  
Запросы выполняются тестовым клиентом Flask, вход –  
настоящими учётными записями PostgreSQL."""  
import csv  
import html  
import os  
import re  
import sys  
import sys  
import textwrap  
import time  
from html.parser import HTMLParser  
from pathlib import Path  
  
import psycopg  
  
from webapp import create_app  
  
OUT = Path(sys.argv[1] if len(sys.argv) > 1  
            else "verification")  
PW = {  
    "reception_orlova": os.environ.get(  
        "IBBD_RECEPTION_PASSWORD", "reception-demo-pass"),  
    "restorer_smirnov": os.environ.get(  
        "IBBD_RESTORER_PASSWORD", "restorer-demo-pass"),  
    "workshop_admin": os.environ.get(  
        "IBBD_ADMIN_PASSWORD", "admin-demo-pass"),  
}
```

```

NEW_PW = "new-user-demo-pass"
PG = dict(host="127.0.0.1",
          port=int(os.environ.get("IBBD_PGPORT", "55432")),
          dbname="workshop")
results = []

```

```

class Cells(HTMLParser):
    """Строки всех таблиц HTML-страницы: списки ячеек."""

    def __init__(self, page):
        super().__init__()
        self.rows, self.row, self.cell = [], None, None
        self.feed(page)

    def handle_starttag(self, tag, attrs):
        if tag == "tr":
            self.row = []
        elif tag in ("td", "th") and self.row is not None:
            self.cell = ""

    def handle_endtag(self, tag):
        if tag in ("td", "th") and self.cell is not None:
            self.row.append(" ".join(self.cell.split()))
            self.cell = None
        elif tag == "tr" and self.row is not None:
            self.rows.append(self.row)
            self.row = None

    def handle_data(self, data):
        if self.cell is not None:
            self.cell += data

    def text(resp):
        return resp.get_data(as_text=True)

    def flashes(resp):
        found = re.findall(r'class="flash[^"]*">([<]*)<',
                           text(resp))
        return [html.unescape(f) for f in found]

```

```

def check(name, ok, detail=""):
    results.append((ok, name, detail))
    print("ok " if ok else "FAIL", name, detail)

def csrf(client):
    page = text(client.get("/login"))
    m = re.search(r'name="csrf" value="([^\"]+)"', page)
    return m.group(1) if m else None

def token(client, path):
    m = re.search(r'name="csrf" value="([^\"]+)"',
                  text(client.get(path)))
    return m.group(1)

def login(client, user, password):
    return client.post("/login", data={
        "username": user, "password": password,
        "csrf": csrf(client)})

def post(client, path, data, page="/tables"):
    data = dict(data, csrf=token(client, page))
    return client.post(path, data=data,
                       follow_redirects=True)

def save(name, content):
    (OUT / name).write_text(content, encoding="utf-8")

def save_csv(name, rows, drop=()):
    """Результат страницы в CSV для таблиц отчёта."""
    keep = [i for i, h in enumerate(rows[0])
            if h not in drop]
    with open(OUT / name, "w", newline="",
              encoding="utf-8") as f:
        csv.writer(f).writerows(
            [[r[i] for i in keep] for r in rows])

```

```

def main():
    OUT.mkdir(parents=True, exist_ok=True)
    app = create_app({"TESTING": True})
    store = app.extensions["ibbd_store"]

    # 1. Аутентификация.
    c = app.test_client()
    r = login(c, "reception_orlova", "wrong-password")
    check("неверный пароль отклонён", r.status_code == 401,
          f"HTTP {r.status_code}")
    r = login(c, "postgres", "anything")
    check("postgres (trust) не допускается",
          r.status_code == 401, f"HTTP {r.status_code}")
    r = login(c, "no_such_user", "whatever-pass")
    check("несуществующий пользователь",
          r.status_code == 401, f"HTTP {r.status_code}")
    r = c.post("/login", data={
        "username": "reception_orlova",
        "password": PW["reception_orlova"]})
    check("вход без CSRF-токена", r.status_code == 400,
          f"HTTP {r.status_code}")
    c2 = create_app({"TESTING": True}).test_client()
    codes = [login(c2, "restorer_volkova", "bad-pass-"
                  + str(i)).status_code for i in range(5)]
    r = login(c2, "restorer_volkova",
              PW["restorer_smirnov"])
    check("блокировка после 5 неудач",
          "Слишком много" in text(r),
          f"HTTP {codes[-1]}, затем {r.status_code}")

    clients = {}
    for user, pw in PW.items():
        cl = app.test_client()
        r = login(cl, user, pw)
        check(f"вход {user}", r.status_code == 302,
              f"HTTP {r.status_code}")
        clients[user] = cl
    rec = clients["reception_orlova"]
    res = clients["restorer_smirnov"]
    adm = clients["workshop_admin"]

    r = rec.get("/tables")

```

```

h = r.headers
check("заголовки безопасности",
      "default-src 'self'" in h["Content-Security-Policy"]
      and h["X-Frame-Options"] == "DENY"
      and h["Cache-Control"] == "no-store")
cookie = rec.get_cookie("session")
check("cookie HttpOnly, SameSite=Strict",
      cookie.http_only and cookie.same_site == "Strict")

# 2. Списки отношений по ролям.
counts = {}
for user, cl in clients.items():
    rows = Cells(text(cl.get("/tables"))).rows
    rows = [r for r in rows if len(r) == 6]
    counts[user] = len(rows) - 1
    save_csv(f"tables-{user}.csv", rows)
check("отношений у приёмщика", counts[
    "reception_orlova"] == 8,
      str(counts["reception_orlova"]))
check("отношений у реставратора", counts[
    "restorer_smirnov"] == 9,
      str(counts["restorer_smirnov"]))
check("отношений у администратора", counts[
    "workshop_admin"] == 15,
      str(counts["workshop_admin"]))

# 3. Просмотр, сортировка, фильтр, скрытые столбцы.
r = rec.get("/t/customer?fcol=full_name&fop=like"
            "&fval=ова&sort=full_name&dir=desc")
rows = Cells(text(r)).rows
names = [x[1] for x in rows[1:]]
check("фильтр и сортировка customer",
      names == sorted(names, reverse=True)
      and all("ова" in n for n in names),
      f"{len(names)} строк")
save_csv("customer-filter.csv",
        [x[:3] for x in rows])
r = res.get("/t/instrument")
check("скрыт instrument.customer_id у реставратора",
      "Скрыты столбцы без права чтения:" in text(r)
      and "customer_id" not in Cells(text(r)).rows[0])
for path in ("/t/payment", "/t/change_log",
            "/t/customer%3B--", "/r/payments"):

```

```

code = res.get(path).status_code
check(f"реставратор {path}", code == 404,
      f"HTTP {code}")
r = rec.get("/t/customer?sort=full_name;DROP"
            "&fcol=phone&fval=' OR 1=1 --")
n = len(Cells(text(r)).rows) - 1
check("попытка SQL-инъекции в фильтре",
      r.status_code == 200 and n == 0, f"{n} строк")

# 4. Вставка, изменение, удаление; решает СУБД.
r = post(rec, "/t/customer/insert", {
    "col_full_name": "Тестова Вера Ивановна",
    "col_phone": "+7 900 000-00-99",
    "col_email": "vera@example.test"})
check("приёмщик добавил заказчика",
      "Добавлено строк: 1" in flashes(r))
new_id = next(x[0] for x in Cells(text(
    rec.get("/t/customer?fcol=full_name&fop=eq"
            "&fval=Тестова Вера Ивановна"))).rows[1:]))
r = post(rec, "/t/customer/delete",
        {"key_customer_id": new_id})
check("приёмщик не может удалить", any(
    "42501" in f for f in flashes(r)), flashes(r)[0])
r = rec.post("/t/customer/insert", data={
    "col_full_name": "X", "col_phone": "1",
    "csrf": "forged"})
check("POST с чужим CSRF-токеном",
      r.status_code == 400, f"HTTP {r.status_code}")
r = post(res, "/t/order_work/update", {
    "key_order_id": "1", "key_line_no": "1",
    "column": "status", "value": "in_progress"})
check("реставратор изменил свою работу",
      "Изменено строк: 1" in flashes(r))
r = post(res, "/t/order_work/update", {
    "key_order_id": "2", "key_line_no": "1",
    "column": "status", "value": "done"})
check("чужая работа не изменена (RLS)",
      "Изменено строк: 0" in flashes(r))
r = post(res, "/t/order_work/update", {
    "key_order_id": "1", "key_line_no": "1",
    "column": "agreed_price_kopecks", "value": "0"})
check("цена недоступна для изменения",
      r.status_code == 400, f"HTTP {r.status_code}")

```

```

r = post(res, "/t/inspection/insert", {
    "col_order_id": "2", "col_employee_id": "2",
    "col_inspected_on": "2026-09-19",
    "col_conclusion": "Подпись чужим именем"})
check("осмотр от чужого имени (RLS)", any(
    "42501" in f for f in flashes(r)), flashes(r)[0])
r = post(res, "/t/inspection/insert", {
    "col_order_id": "1", "col_employee_id": "1",
    "col_inspected_on": "2026-09-19",
    "col_conclusion": "Проверка через веб-интерфейс"})
check("осмотр от своего имени",
    "Добавлено строк: 1" in flashes(r))
r = post(adm, "/t/customer/delete",
    {"key_customer_id": new_id})
check("администратор удалил заказчика",
    "Удалено строк: 1" in flashes(r))
r = post(adm, "/t/customer/delete",
    {"key_customer_id": "1"})
check("удаление связанной строки (FK)", any(
    "23503" in f for f in flashes(r)), flashes(r)[0])

```

5. Сводные отчёты с GROUP BY.

```

specs = [
    (rec, "payments", "?sort=paid_rub&dir=desc", ()),
    (res, "workload",
        "?ostatus=&sort=works&dir=desc", ()),
    (adm, "employees",
        "?sort=agreed_rub&dir=desc", ()),
    (adm, "audit", "?sort=total&dir=desc",
        ("app_group", "last_change")),
]
for cl, key, qs, drop in specs:
    rows = Cells(text(cl.get("/r/" + key + qs))).rows
    check(f"отчёт {key}", len(rows) > 1,
        f"{len(rows) - 1} групп")
    save_csv(f"report-{key}.csv", rows, drop)
rows = Cells(text(rec.get(
    "/r/payments?method=card&fcol=instrument_type"
    "&fval=к&sort=instrument_type"))).rows
check("отчёт payments: фильтр и сортировка",
    len(rows) > 1 and all(
        "к" in x[0].lower() for x in rows[1:]),
    f"{len(rows) - 1} групп")

```

```

save_csv("report-payments-card.csv", rows)
rows = Cells(text(res.get(
    "/r/workload?mine=1&sort=service"))).rows
check("отчёт workload: только мои",
    len(rows) > 1 and all(
        x[0].startswith("Смирнов") for x in rows[1:]),
    f"{len(rows) - 1} групп")
save_csv("report-workload-mine.csv", rows)
r = rec.get("/r/audit")
check("приёмщику отчёт audit недоступен",
    r.status_code == 404, f"HTTP {r.status_code}")

# 6. Регистрация пользователей.
r = post(adm, "/register", {
    "login": "reception_petrova",
    "group": "reception_role",
    "password": NEW_PW}, "/register")
check("регистрация reception_petrova",
    "Создан пользователь reception_petrova"
    in flashes(r))
r = post(adm, "/register", {
    "login": "restorer_kozlov",
    "group": "restorer_role", "employee_id": "3",
    "password": NEW_PW}, "/register")
check("регистрация restorer_kozlov",
    "Создан пользователь restorer_kozlov"
    in flashes(r))
r = post(adm, "/register", {
    "login": "hacker", "group": "admin_role",
    "password": NEW_PW}, "/register")
check("регистрация в admin_role отклонена", any(
    "недопустимая роль" in f for f in flashes(r)),
    flashes(r)[0])
code = rec.get("/register").status_code
check("приёмщику регистрация недоступна",
    code == 403, f"HTTP {code}")
rows = Cells(text(adm.get("/register"))).rows
save_csv("users.csv", rows)
for user in ("reception_petrova", "restorer_kozlov"):
    cl = app.test_client()
    r = login(cl, user, NEW_PW)
    check(f"вход {user}", r.status_code == 302,
        f"HTTP {r.status_code}")

```

```

r = post(cl, "/t/order_work/update", {
    "key_order_id": "4", "key_line_no": "1",
    "column": "status", "value": "done"})
check("restorer_kozlov изменил свою работу",
    "Изменено строк: 1" in flashes(r))

with psycopg.connect(**PG, user="reception_orlova",
    password=PW["reception_orlova"],
    autocommit=True) as conn:
    try:
        conn.execute("SELECT app_register_user("
            "'x_user', 'reception_role', "
            "'SCRAM-SHA-256$4096:x')")
        check("функция регистрации закрыта", False)
    except psycopg.errors.InsufficientPrivilege as e:
        check("функция регистрации закрыта", True,
            e.sqlstate)
with psycopg.connect(**PG, user="postgres") as conn:
    rows = conn.execute(
        "SELECT rolname, left(rolpassword, 19)"
        " FROM pg_authid WHERE rolname IN"
        " ('reception_petrova', 'restorer_kozlov',"
        " 'workshop_admin') ORDER BY 1").fetchall()
    check("пароли хранятся как SCRAM", all(
        p == "SCRAM-SHA-256$4096:" for _, p in rows))
    save_csv("scram.csv",
        [{"rolname", "rolpassword"}]
        + [list(x) for x in rows])

# 7. Журнал и завершение сеанса.
rows = Cells(text(adm.get(
    "/t/change_log?sort=log_id"))).rows
head = rows[0]
keep = ["log_id", "db_role", "operation",
    "table_name", "row_key"]
log = [[r[head.index(k)] for k in keep] for r in rows]
roles = {r[1] for r in log[1:]}
check("журнал: роли входа из веб-сеансов",
    {"reception_orlova", "restorer_smirnov",
    "workshop_admin", "restorer_kozlov"} <= roles,
    f"{len(log) - 1} записей")
save_csv("change-log.csv", log)

```

```

old = rec.get_cookie("session").value
post(rec, "/logout", {})
rec.set_cookie("session", old)
r = rec.get("/tables")
check("старая cookie после выхода",
      r.status_code == 302, f"HTTP {r.status_code}")
for us in list(store._items.values()):
    if us.login == "restorer_smirnov":
        us.seen = time.time() - 16 * 60
r = res.get("/tables")
check("истечение сеанса (15 мин)",
      r.status_code == 302, f"HTTP {r.status_code}")

failed = [n for ok, n, _ in results if not ok]
summary = (f"Проверок: {len(results)}, успешно: "
          f"{len(results) - len(failed)}")

lines = []
for ok, n, d in results:
    line = f"{'ok' if ok else 'FAIL'} {n}"
    if d and len(line) + len(d) + 3 <= 62:
        lines.append(f"{line} [{d}]")
        continue
    lines.append(line)
    lines += textwrap.wrap(d, 56, initial_indent=" " * 6,
                          subsequent_indent=" " * 6)
save("check.txt", "\n".join(lines + ["", summary])
     + "\n")
print(summary)
return 1 if failed else 0

if __name__ == "__main__":
    sys.exit(main())

```

Листинг Г.2 — Проверка приложения («check.py»)

```

"""Снимки экрана работающего приложения: сервер на
127.0.0.1, браузер Google Chrome под управлением
Playwright входит через форму, как пользователь."""
import logging
import os
import sys
import threading

```

```

from pathlib import Path

from playwright.sync_api import sync_playwright
from werkzeug.serving import make_server

from webapp import create_app

OUT = Path(sys.argv[1] if len(sys.argv) > 1 else "shots")
PW = {
    "reception_orlova": os.environ.get(
        "IBBD_RECEPTION_PASSWORD", "reception-demo-pass"),
    "restorer_smirnov": os.environ.get(
        "IBBD_RESTORER_PASSWORD", "restorer-demo-pass"),
    "workshop_admin": os.environ.get(
        "IBBD_ADMIN_PASSWORD", "admin-demo-pass"),
}

def main():
    logging.getLogger("werkzeug").setLevel(logging.WARNING)
    OUT.mkdir(parents=True, exist_ok=True)
    server = make_server("127.0.0.1", 0, create_app(),
                        threaded=True)
    base = f"http://127.0.0.1:{server.server_port}"
    threading.Thread(target=server.serve_forever,
                    daemon=True).start()
    with sync_playwright() as p:
        browser = p.chromium.launch(channel="chrome")
        ctx = browser.new_context(
            viewport={"width": 1100, "height": 700},
            locale="ru-RU")
        page = ctx.new_page()

        def shot(name, path=None):
            if path:
                page.goto(base + path)
            page.screenshot(path=OUT / f"{name}.png",
                           full_page=True)

        def login(user, password):
            page.goto(base + "/login")
            page.fill("input[name=username]", user)
            page.fill("input[name=password]", password)

```

```

        page.click("form.card button")

def logout():
    page.click("header form button")

login("reception_orlova", "wrong-password")
page.screenshot(path=OUT / "01-login-failed.png",
                clip={"x": 0, "y": 0,
                      "width": 1100, "height": 330})
login("reception_orlova", PW["reception_orlova"])
shot("02-reception-tables")
shot("03-reception-customer",
      "/t/customer?fcol=full_name&fop=like"
      "&fval=oba&sort=full_name&dir=desc")
shot("04-reception-report",
      "/r/payments?sort=paid_rub&dir=desc")
logout()

login("restorer_smirnov", PW["restorer_smirnov"])
shot("05-restorer-tables")
shot("06-restorer-instrument", "/t/instrument")
page.goto(base + "/t/inspection")
form = "form[action$='/insert'] "
page.fill(form + "[name=col_order_id]", "2")
page.fill(form + "[name=col_employee_id]", "2")
page.fill(form + "[name=col_inspected_on]",
          "2026-09-19")
page.fill(form + "[name=col_conclusion]",
          "Подпись чужим именем")
page.click(form + "button")
page.screenshot(path=OUT / "07-restorer-rls.png",
                clip={"x": 0, "y": 0,
                      "width": 1100, "height": 330})
shot("08-restorer-report",
      "/r/workload?sort=works&dir=desc")
logout()

login("workshop_admin", PW["workshop_admin"])
shot("09-admin-tables")
shot("10-admin-audit",
      "/r/audit?sort=total&dir=desc")
shot("11-admin-users", "/register")
logout()

```

```
        browser.close()
    server.shutdown()
    print("ok:", OUT)

if __name__ == "__main__":
    main()
```

Листинг Г.3 — Снимки экрана («shots.py»)